



Cyberscope

A *TAC Security* Company

Audit Report

MemeBitcoin

September 2026

Network : BSC

Token (Proxy): 0x3b93314430A7db7d614eB3A6c226dEo01973d0Ef

Audited Implementation: 0x54CB3dD5284041Bd1207be034AF95cCcBF71EF22

Audited by © cyberscope

Table of Contents

Table of Contents	1
Analysis	2
Diagnostics	3
Risk Classification	4
Review	5
Audit Updates	5
Findings Breakdown	6
CCR - Contract Centralization Risk	7
Description	7
Recommendation	7
Team Update	7
RIC - Redundant Inherited Contract	8
Description	8
Recommendation	8
Team Update	8
RSE - Release Schedule Exhaustion	9
Description	9
Recommendation	9
Team Update	9
L04 - Conformance to Solidity Naming Conventions	10
Description	10
Recommendation	11
Team Update	11
L16 - Validate Variable Setters	12
Description	12
Recommendation	12
Team Update	12
L19 - Stable Compiler Version	13
Description	13
Recommendation	13
Team Update	13
Functions Analysis	14
Inheritance Graph	15
Flow Graph	16
Summary	17
Disclaimer	18
About Cyberscope	19

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Passed
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	CCR	Contract Centralization Risk	Acknowledged
●	RIC	Redundant Inherited Contract	Acknowledged
●	RSE	Release Schedule Exhaustion	Acknowledged
●	L04	Conformance to Solidity Naming Conventions	Acknowledged
●	L16	Validate Variable Setters	Acknowledged
●	L19	Stable Compiler Version	Acknowledged

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

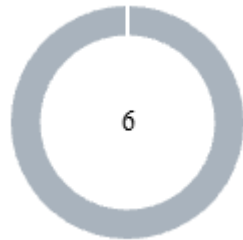
Review

Token (Proxy)	https://bscscan.com/address/0x3b93314430A7db7d614eB3A6c226dEa01973d0Ef
Audited Implementation	https://bscscan.com/address/0x54cb3dd5284041bd1207be034af95cccbf71ef22
Decimals	8

Audit Updates

Initial Audit	21 Sep 2026
----------------------	-------------

Findings Breakdown



● Critical	0
● Medium	0
● Minor / Informative	6

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	0
● Medium	0	0	0	0
● Minor / Informative	0	6	0	0

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	contracts/MbtcBscTokenUpgradable.sol#L140,153
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function setDistributionContract(address _newContract) external onlyOwner
{...}

function _authorizeUpgrade(address newImplementation) internal view
override onlyOwner {...}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

The team has acknowledged that this is not a security issue and states:

The owner is a 2-of-3 Safe multisig. Its authority is limited to changing the emission destination and upgrading the implementation contract.

RIC - Redundant Inherited Contract

Criticality	Minor / Informative
Location	contracts/MbtcBscTokenUpgradable.sol#L8,22
Status	Acknowledged

Description

The contract inherits `AccessControlUpgradeable` but does not assign any role, including the `DEFAULT_ADMIN_ROLE`, and does not use roles to restrict any of its functions. As a result, the role management functionality cannot be utilized by any account and the inherited code increases the contract size without serving the application logic.

```
AccessControlUpgradeable
```

Recommendation

The team is advised to reconsider the inheritance of the `AccessControlUpgradeable` contract and ensure that it is properly configured if role-based access control is intended.

Team Update

The team has acknowledged that this is not a security issue and states:

No address currently holds the relevant role, and under the current contract structure the role cannot be assigned in the future, so there is no effective privilege associated with it.

RSE - Release Schedule Exhaustion

Criticality	Minor / Informative
Location	contracts/MbtcBscTokenUpgradable.sol#L117,128
Status	Acknowledged

Description

The `releaseTokens` function halves the `currentReleaseAmount` at the end of every halving cycle and requires the amount to be greater than zero. Since integer division is used, the amount will eventually reach zero while the contract still holds a positive balance. From that point the function reverts on every call and the remaining balance cannot be released through the schedule.

```
require(amountToRelease > 0, "No tokens left to release");  
currentReleaseAmount = currentReleaseAmount / 2;
```

Recommendation

The team is advised to reconsider the handling of the remaining balance once the calculated release amount reaches zero.

Team Update

The team has acknowledged that this is not a security issue and states:

The remaining balance is approximately 0.005 MBTC, which has no material impact.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	contracts/MbtcBscTokenUpgradable.sol#L39,65,66,140
Status	Acknowledged

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
uint256 public HALVING_RELEASES_PER_CYCLE;
address _initialOwner
address _distributionContract
function setDistributionContract(address _newContract) external
onlyOwner {
    ...
}
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

Team Update

The team has acknowledged that this is not a security issue and states:

This is a labeling/naming issue and does not affect the actual behavior or execution of the deployed contract.

L16 - Validate Variable Setters

Criticality	Minor / Informative
Location	contracts/MbtcBscTokenUpgradable.sol#L74
Status	Acknowledged

Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing of proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

```
distributionContract = _distributionContract
```

Recommendation

By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

Team Update

The team has acknowledged that this is not a security issue and states:

Initialization has already been completed, and the implementation contract's initializer is locked, preventing it from being initialized again.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	contracts/MbtcBscTokenUpgradable.sol#L2
Status	Acknowledged

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
pragma solidity ^0.8.20;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

Team Update

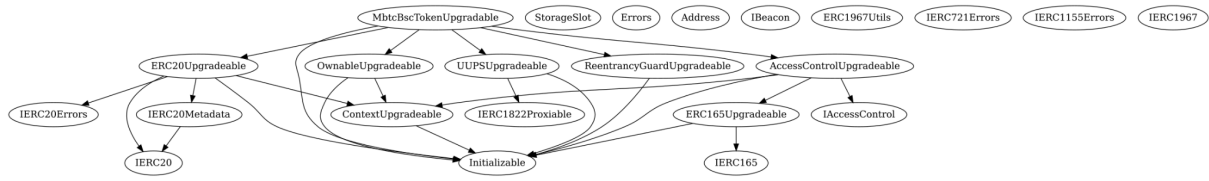
The team has acknowledged that this is not a security issue and states:

The Solidity compiler version is fixed at 0.8.24, which is consistent with the verified source code information shown on BscScan.

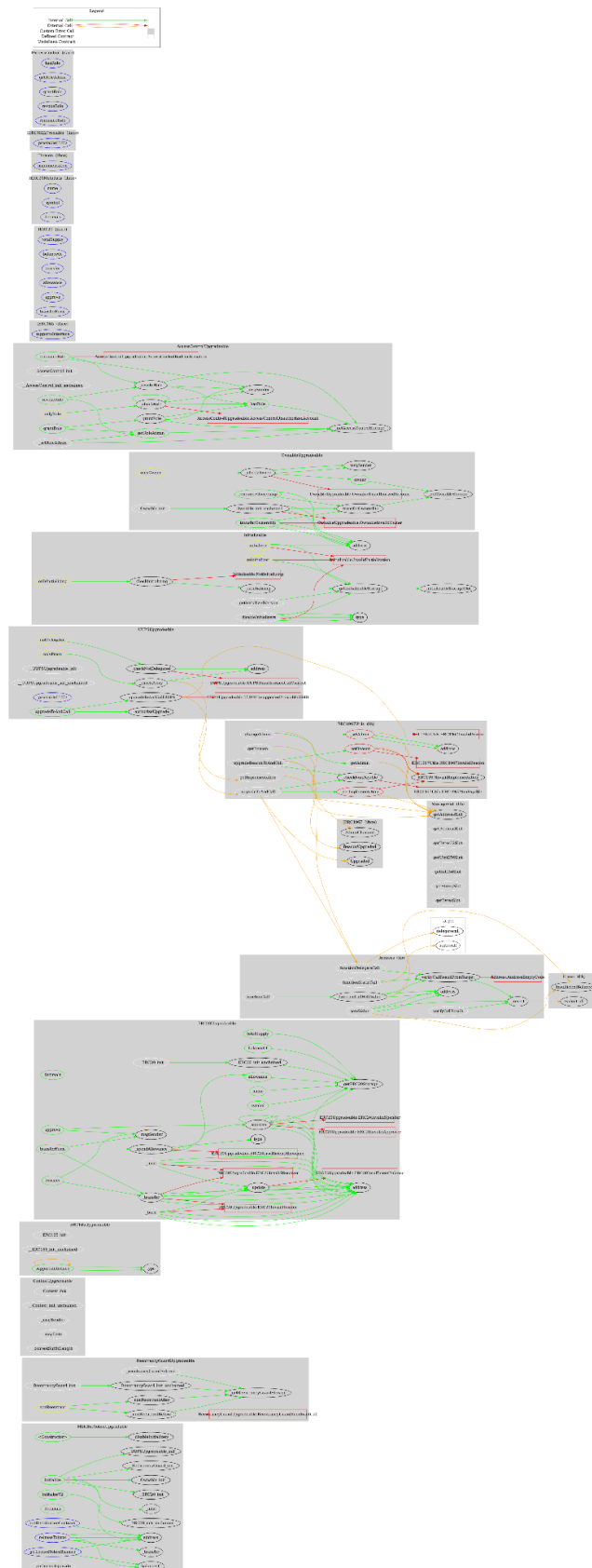
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
MbtcBscToken Upgradable	Implementation	Initializable, ERC20Upgr deable, OwnableUpg radeable, ReentrancyG uardUpgrade able, AccessContr olUpgradeab le, UUPSUpgra deable		
		Public	✓	-
	initialize	Public	✓	initializer
	initializeV2	Public	✓	reinitializer
	decimals	Public		-
	releaseTokens	External	✓	nonReentrant
	setDistributionContract	External	✓	onlyOwner
	getLockedTokenBalance	External		-
	_authorizeUpgrade	Internal		onlyOwner

Inheritance Graph



Flow Graph



Summary

MemeBitcoin contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. MemeBitcoin is an interesting project that has a friendly and growing community. The Smart Contract analysis reported no compiler error or critical issues. The contract Owner (2-of-3 Safe multisig with address 0x0Bc23BaB9B36c9fca36965205c51d2b32e901fF7) can access some admin functions that can not be used in a malicious way to disturb the users' transactions.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io